Ember Attractor

Niagara at Scale | One-page proposal | UE 5.7 | Individual project

Direction and scope

An independent, one-room particle-control experiment inspired by the gravity-manipulation interaction in our Gravity project. The player guides explosion embers with a movable attraction field. This isolates a potential future interaction and VFX direction without importing assets, modifying Gravity, or claiming an existing integration. This is a proposed custom standalone variation; instructor approval is requested because the published Track B specifically describes boids.

Mechanic

A repeatable burst releases hot particles into a test chamber. Moving the mouse positions an attraction field; holding the mouse button draws particles toward it. The intended completed mechanic is to guide hot material into a recovery region before it cools. Recovery must depend on simulated particle state, rather than a separate invisible overlap proxy. Week 1 establishes both simulations, input control, count control, and the first performance capture; end-to-end scoring and GPU-to-gameplay feedback are Week 2 work.

Two ends of the cost curve

Simple and many: 100,000+ GPU sprite embers using attraction and drag. **Complex and few:** 500-5,000 CPU mesh fragments with movement and collision response, adding per-particle thermal state and recovery events in Week 2. Both use the same attraction interaction and independent runtime particle-count controls. Additional complex behavior and particle-to-gameplay events will be completed and evaluated in Week 2.

Gameplay data path

C++ supplies count, attraction position, enable state, and strength through Niagara user parameters. CPU simulation output will be exported to C++ for recovery events. For GPU feedback, prototype a compact occupancy/heat accumulation grid with asynchronous readback; measure latency and overhead, and document any approximation. Neither exposed parameters nor a visual attraction effect alone proves two-way coupling.

Budget and measurement

Target 1080p at 60 fps (16.6 ms/frame) on lab hardware, with combined Niagara game/render-thread cost at or below 4 ms; record GPU cost separately. Compare baseline, GPU-only, CPU-only, and combined modes at a fixed camera. Sweep GPU counts 10k/25k/50k/100k/200k and CPU counts 500/1k/2k/3.5k/5k, extending until the budget fails. Preserve Unreal Insights traces, CSV captures, and stat/debugger evidence. Week 3 will measure a controlled optimization, such as an Effect Type scalability policy, against the same workload.

References and milestones

Primary mechanical reference: PixelJunk Shooter Ultimate, where controlling simulated material drives puzzles. **Secondary: Noita**, for hot-material readability and material-state consequences. Official design sources are linked below; personal play/watch notes must be added after reviewing at least two games, as the assignment requests.

Week 1: both systems in a test map, runtime count controls, first capture. **Week 2:** playable recovery loop, verified two-way feedback, complete scaling sweep. **Week 3:** measured optimization, two-page report, 60-90 second video, tagged repository release.

Sources: Noita official site · PixelJunk Shooter Ultimate developer article